universidade de aveiro

Dispositivos Conectados

Interfaces de comunicação local

Paulo C. Bartolomeu

Interfaces de comunicação local: porque são importantes?

- **Integração de sensores e atuadores**: permitem ligar múltiplos dispositivos ao mesmo microcontrolador.
- **Eficiência energética**: protocolos leves e adequados a sistemas de baixo consumo.
- **Redução de custos**: utilização de interfaces já suportadas pelo hardware, sem necessidade de componentes adicionais complexos.
- **Flexibilidade de projeto**: escolha da interface mais adequada consoante requisitos de velocidade, distância e número de dispositivos.
- **Escalabilidade**: suporte à expansão do sistema com novos módulos sem grandes alterações no hardware.
- **Confiabilidade**: protocolos testados e amplamente utilizados, com suporte direto em SDKs como o ESP-IDF.

Comparação de Interfaces de Comunicação Locais

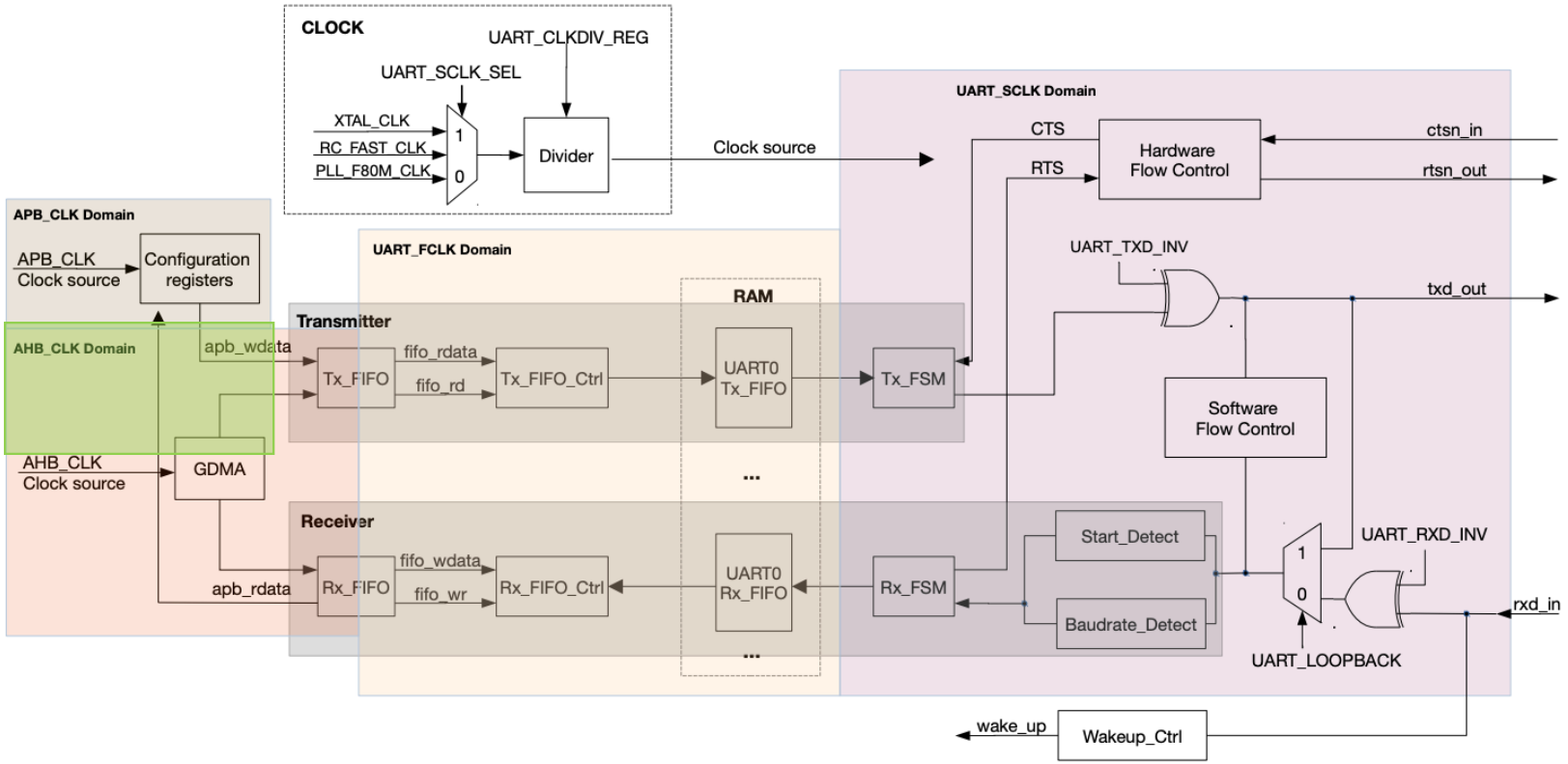
Interface	Nº de ligações	Velocidade	Nº de dispositivos	Complexidade	Uso comum
UART	2 (TX, RX)	Até alguns Mbps	2 (ponto-a-ponto)	Baixa	Comunicação simples entre dois dispositivos (ex.: debug, módulos GPS, Bluetooth clássico)
SPI	4+ (MISO, MOSI, SCLK, CS)	Dezenas de Mbps	1 mestre + vários escravos (cada um com CS)	Média	Sensores de alta velocidade, memória flash, ecrãs TFT
I2C	2 (SDA, SCL)	Até 400 kHz (até alguns MHz em versões rápidas)	1 mestre + até 127 escravos	Alta (endereçamento e protocolo mais complexo)	Sensores de baixo custo, RTCs, expansores de I/O

ESP32-C6 UART

UART Feature	LP_UART Feature
Programmable baud rate up to 5 MBaud	
128 x 8 bit RAM respectively for the TX channel and RX channel of a UART controller	32 x 8-bit RAM respectively for the TX channel and RX channel of LP_UART
Full-duplex asynchronous communication	
Data bits (5 to 8 bits)	
Stop bits (1, 1.5 or 2 bits)	
Parity bit	

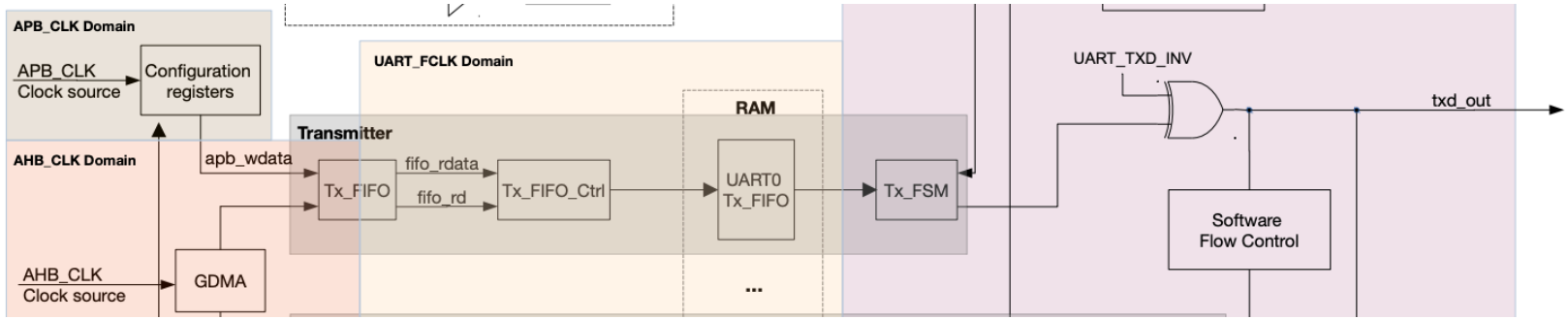
- O ESP32-C6 possui **três controladores UART**, incluindo duas UARTs regulares e uma LP UART de baixo consumo.
- Estas UARTs são compatíveis com vários dispositivos UART e suportam comunicação IrDA (Infrared Data Association) e RS485.
- Uma UART é capaz de estabelecer uma **ligação de dados orientada a caracteres** para **comunicação assíncrona** entre dispositivos.
- A comunicação **não adiciona sinais de relógio** aos dados enviados.
  - Para que a comunicação seja bem-sucedida, o transmissor e o receptor devem **operar na mesma taxa de transmissão, com os mesmos bits de *stop* e bit de paridade**

Estrutura da UART



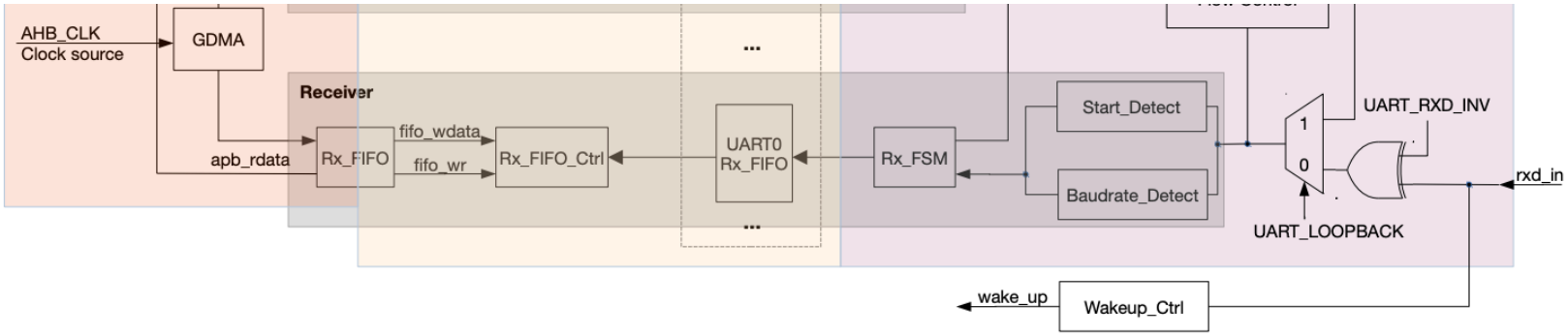
universidade de aveiro

Estrutura da UART: Transmissor



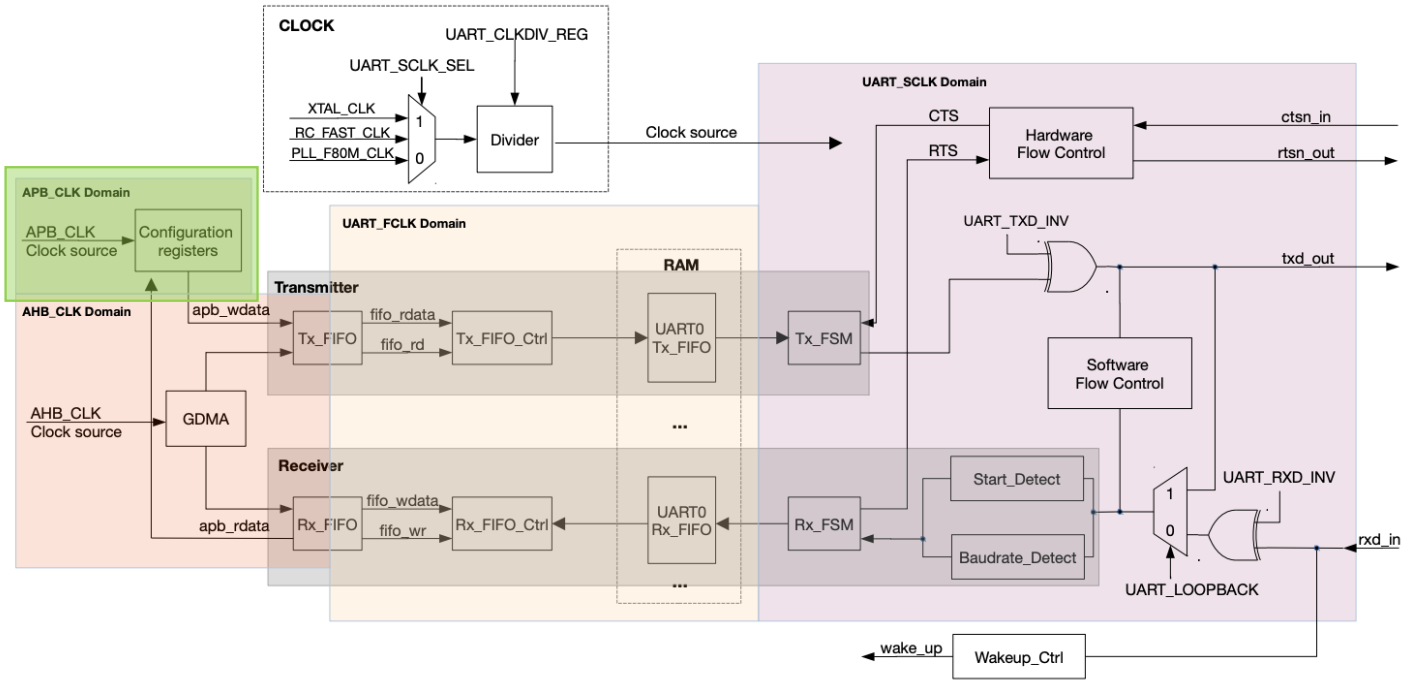
- Contém um **TX FIFO que armazena os dados** a serem enviados. O software pode escrever dados no Tx_FIFO através do barramento APB (Advanced Peripheral Bus), ou mover dados para o Tx_FIFO usando o GDMA (Generic DMA).
- O **Tx_FIFO_Ctrl controla a escrita e leitura do Tx_FIFO**. Quando o Tx_FIFO não está vazio, o Tx_FSM (Finite State Machine do Transmissor) lê os bits de dados na trama de dados através do Tx_FIFO_Ctrl e converte-os numa sequência de bits.
- Os níveis do sinal de sequência de bits de saída (txd_out) podem ser invertidos ao configurar o campo UART_TXD_INV.

Estrutura da UART: Receptor



- **Contém um RX FIFO** que armazena os dados a serem processados.
- O sinal de sequência de bits de entrada (rxd_in) é transferido para a controladora UART, e o seu nível pode ser invertido ao configurar o campo UART_RXD_INV.
- O **Baudrate_Detect mede a taxa de baud do sinal de sequência de bits de entrada** (rxd_in) ao detetar a sua largura mínima de impulso.
- O Start_Detect deteta o bit de início numa trama de dados. Se o bit de início for detetado, o Rx_FSM (Finite State Machine do Recetor) armazena os bits de dados da trama no Rx_FIFO através do Rx_FIFO_Ctrl.
- O **software pode ler dados do Rx_FIFO através do barramento APB ou receber dados usando o GDMA**.

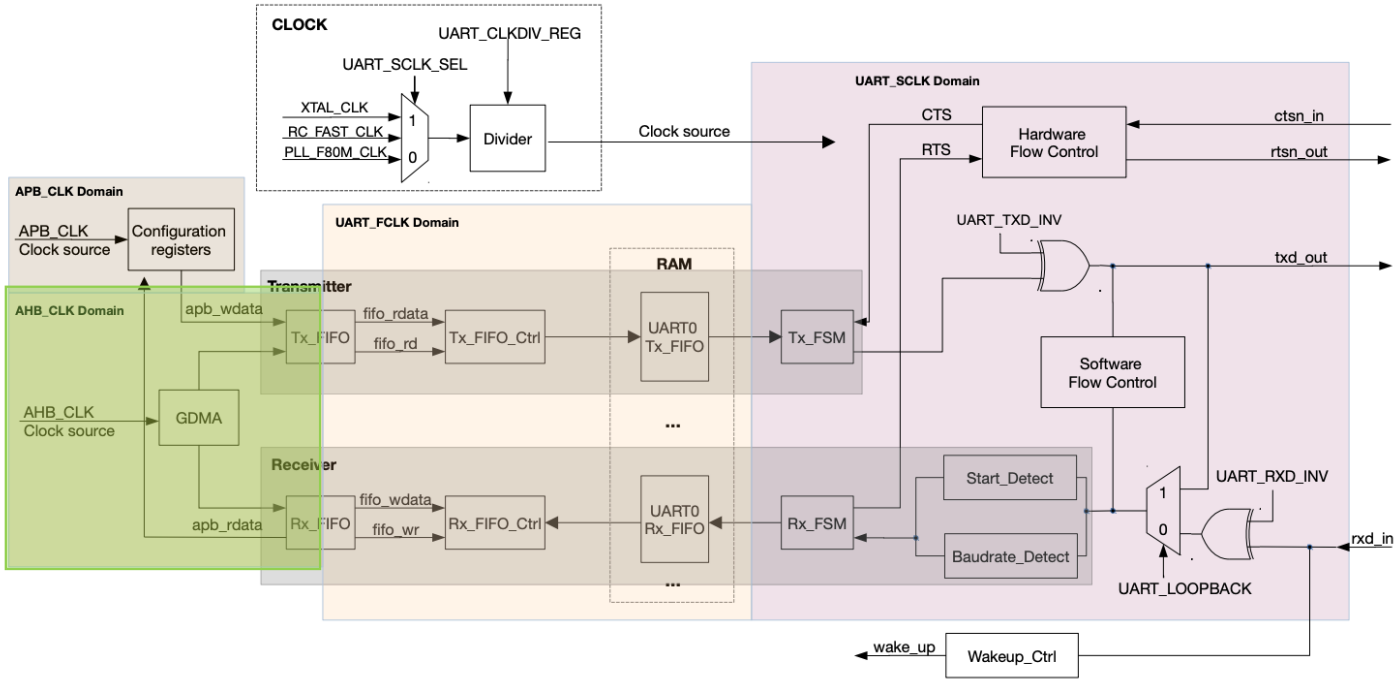
Domínios de relógio



APB_CLK (Advanced Peripheral Bus Clock)

- Relógio principal usado pelos periféricos ligados ao barramento APB.
- No ESP32-C6, é geralmente derivado do relógio do sistema.
- Usado para registos de controlo e configuração da UART.

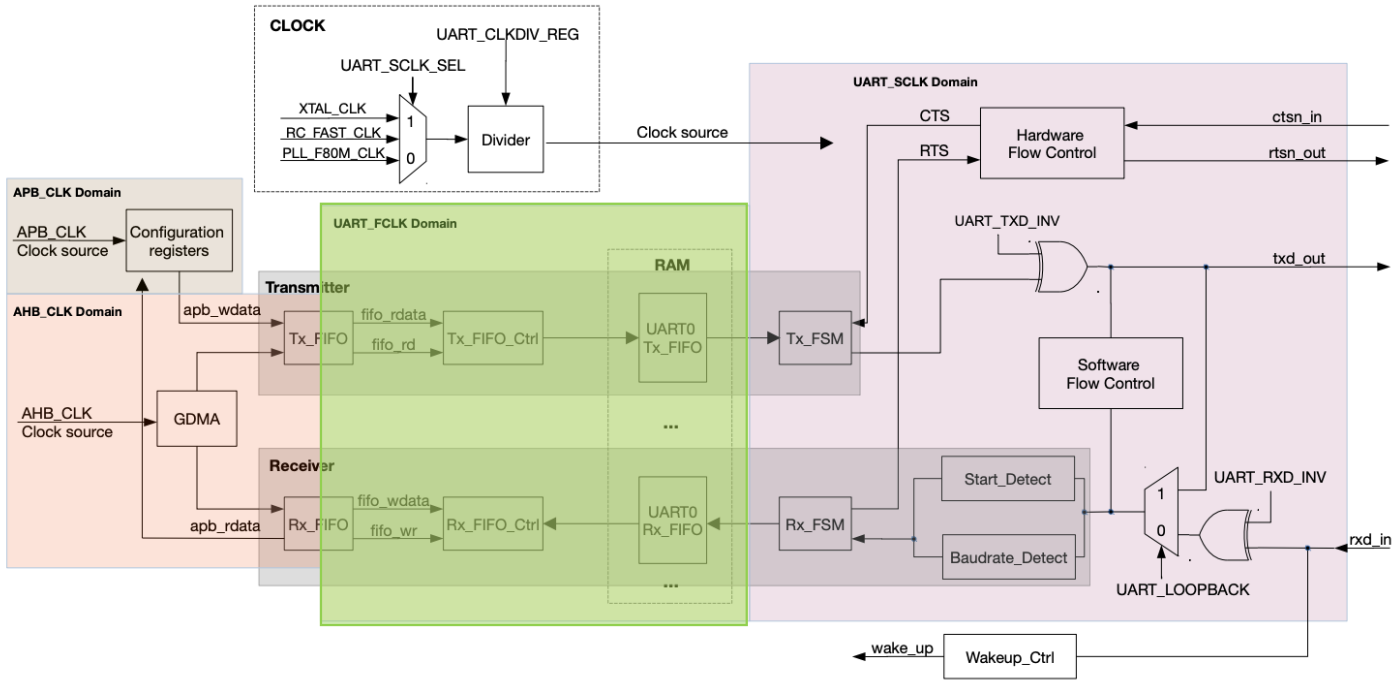
Domínios de relógio



AHB_CLK (Advanced High-Performance Bus Clock)

- Relacionado com o AHB *bus*, que liga periféricos de maior desempenho (DMA, memória, etc.).
- Usado para sincronização entre UART e outros subsistemas com acesso a memória.

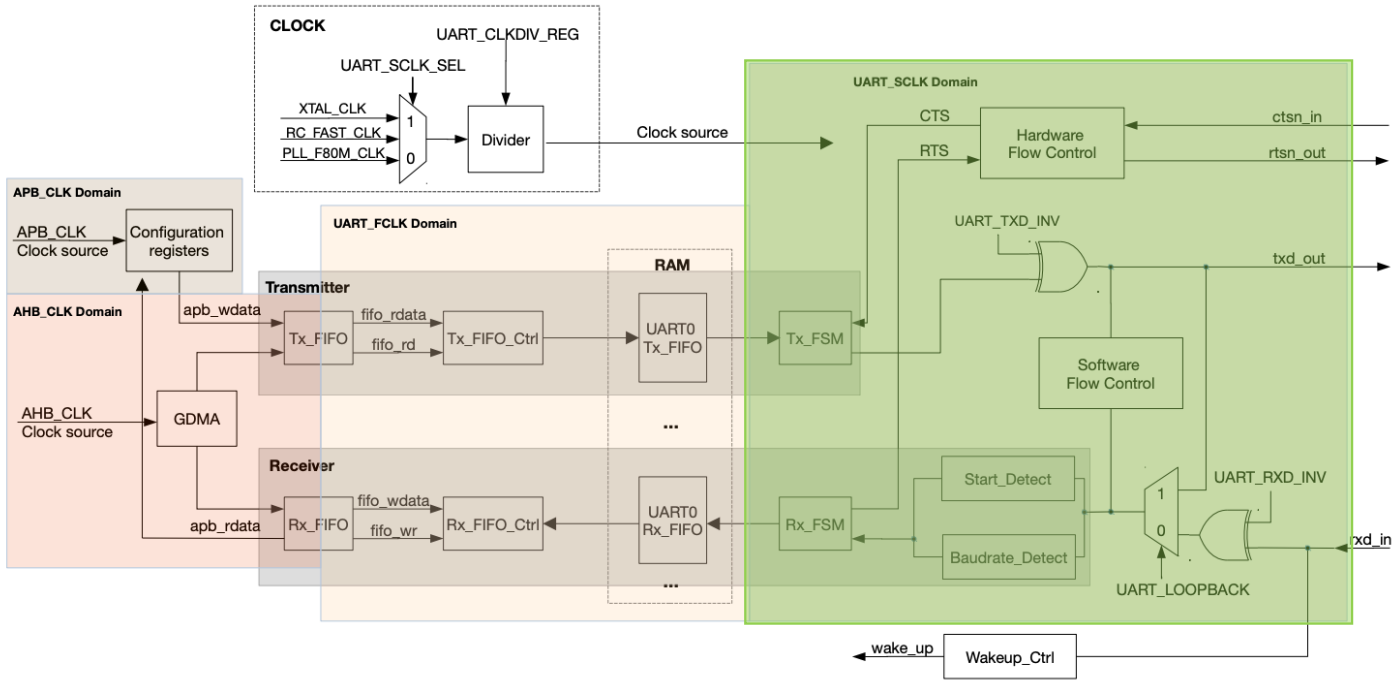
Domínios de relógio



UART_FCLK (UART Function Clock ou Fast Clock)

- Relógio interno de alta frequência usado pelas máquinas de estado da UART para processar os sinais (TX/RX, detetar bits de start/stop, etc.).
- Garante a temporização precisa na receção/transmissão..

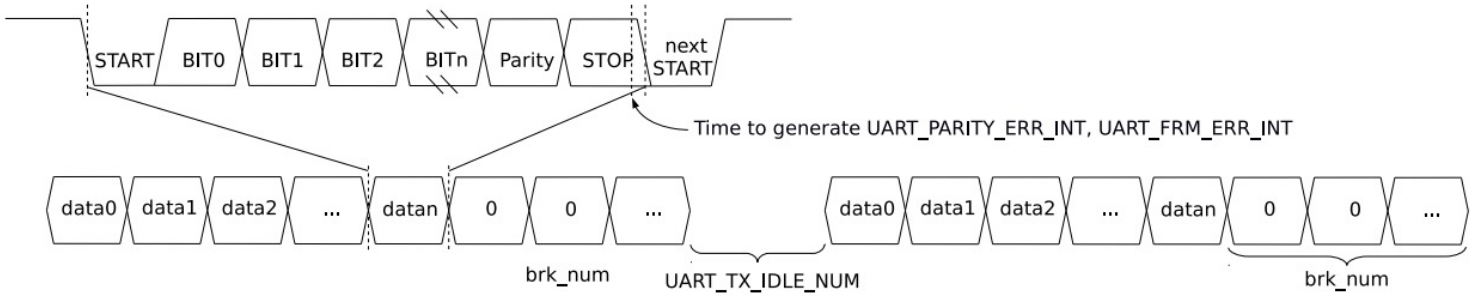
Domínios de relógio



UART_SCLK (UART Source Clock)

- Fonte configurável do relógio que gera a baud rate da UART.
- Pode vir de diferentes fontes internas (XTAL, PLL, etc.).
- Permite ajustar a frequência de operação da UART conforme necessário.

Trama UART



- Uma **trama de dados UART** começa com um bit de início, seguido por bits de dados, um bit de paridade (opcional) e um ou mais bits de stop.
- Os controladores UART no ESP32-C6 suportam vários comprimentos de bits de dados e bits de stop.
- Esses controladores também suportam controle de fluxo de software e hardware, bem como GDMA para transferência de dados em alta velocidade

universidade de aveiro

Erros de comunicação na UART

Overrun Error

- Ocorre quando a UART do recetor recebe um novo byte completo e o armazena no seu buffer (ou FIFO) antes que o software ou DMA tenha lido e processado o byte anterior.

- **Causas**
  - O CPU (ou o sistema) está muito ocupado com outras tarefas (ou a rotina de interrupção é muito lenta) e não leu o byte a tempo do registo/FIFO da UART, resultando na perda do novo byte de entrada.

- **Consequências**
  - Um ou mais bytes de dados são perdidos e a comunicação fica dessincronizada.

Erros de comunicação na UART

Framing Error

- Ocorre quando o recetor não deteta o Bit de Paragem (que deve ser '1', ou nível lógico alto) na posição exata onde a trama de dados deveria terminar. Em vez disso, o recetor lê um '0' (nível lógico baixo).

- **Causas**
  - Baudrates Incompatíveis: A taxa de *baud* (velocidade) do transmissor é ligeiramente diferente da do recetor. Ao longo dos 8-10 bits da trama, a diferença de tempo acumula-se (*clock drift*), fazendo com que o recetor amostre o Bit de Paragem no momento errado.
  - Ruído na Linha: Interferência que inverte o sinal do Bit de Paragem.

- **Consequências**
  - O byte recebido é considerado inválido e deve ser descartado.

Erros de comunicação na UART

Parity Error

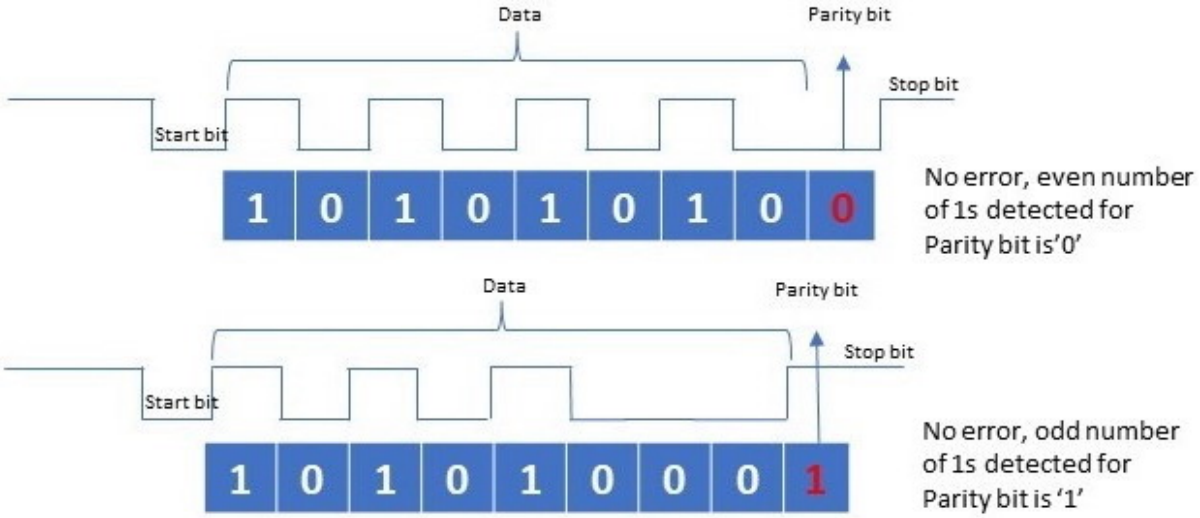
- Ocorre quando o recetor calcula a paridade (o número de bits '1') dos bits de dados recebidos e verifica que não corresponde ao valor do Bit de Paridade que foi transmitido.

- **Causas**
  - Ruído ou Interferência: Algum tipo de ruído na linha causou a inversão de um único bit (de '1' para '0' ou vice-versa) nos bits de dados ou no próprio Bit de Paridade.

- **Consequências**
  - O byte contém um erro de bit e deve ser descartado.

  Nota: A paridade só deteta um número ímpar de erros de bit; se dois bits forem invertidos, o erro de paridade é mascarado e não é detetado.

UART: bit de paridade



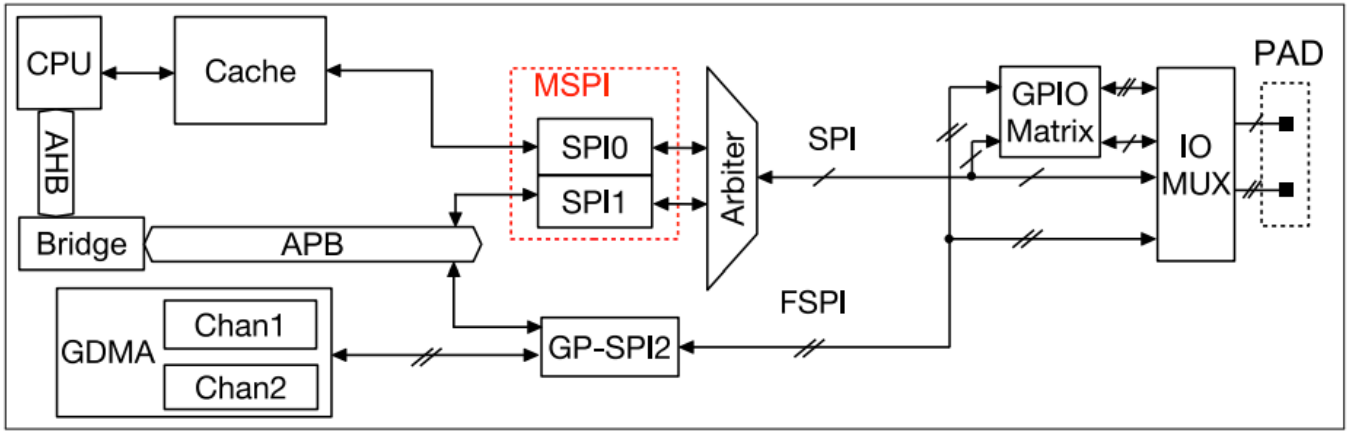
universidade de aveiro

Kahoot! Time

ESP32-C6 SPI

- O módulo ESP32-C6 integra três controladoras SPI:
  - SPI0
  - SPI1
  - **SPI2 de Propósito Geral (GP-SPI2)**
- As controladoras SPI0 e SPI1 (MSPI) estão primariamente reservadas para uso interno a fim de comunicar com a memória flash e SRAM externas.

Arquitetura do Módulo SPI



O GP-SPI2 troca dados com dispositivos SPI das seguintes formas:

- **Transferência controlada pelo CPU**: CPU <-> GP-SPI2 <-> dispositivos SPI
- **Transferência controlada por DMA**: GDMA <-> GP-SPI2 <-> dispositivos SPI
- Os sinais do barramento FSPI são encaminhados para os pinos GPIO através da matriz GPIO ou IO MUX.

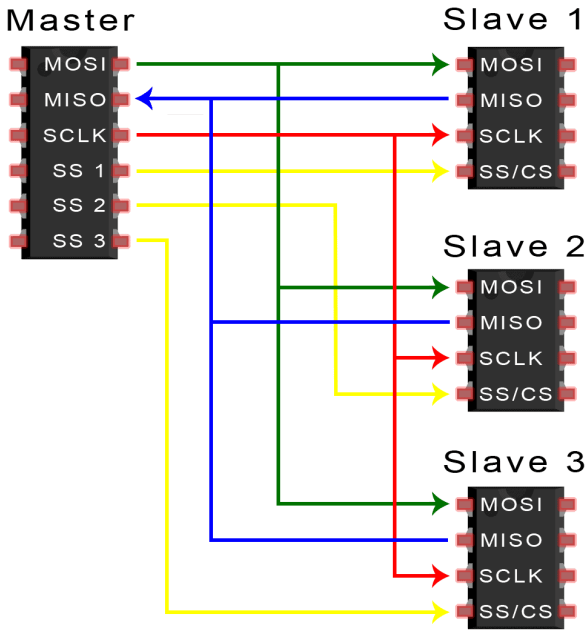
universidade de aveiro

Troca de dados SPI

- **Mestre e Escravo**: O ESP32-C6 (o Mestre) controla a comunicação, definindo a velocidade do relógio (SCLK) e iniciando a comunicação. O periférico externo é o Escravo.
- **Seleção do Escravo**: Para iniciar a comunicação com um dispositivo específico, o Mestre deve puxar a linha CS (*Chip Select*) desse Escravo para o nível lógico baixo (ativo). Todos os outros Escravos na linha mantêm-se inativos.
- **Transferência Full-Duplex**: Uma das maiores vantagens do SPI é que a transferência é full-duplex, ou seja, envio e receção de dados ocorrem ao mesmo tempo (simultaneamente):
- A cada ciclo do SCLK, o Mestre envia um bit pelo MOSI para o Escravo e, no mesmo ciclo, o Escravo envia um bit de volta para o Mestre pelo MISO.
- Para o Mestre **ler um byte do Escravo**, ele deve, obrigatoriamente, **enviar um byte** (muitas vezes um byte "fantasma" ou um comando de leitura) para o Escravo.

SPI Multi-Slave

- O ESP32-C6 (usando o GP-SPI2) **pode ligar-se a vários Escravos** (até seis linhas CS dedicadas).
- Os **sinais SCLK, MOSI e MISO são partilhados** e ligam-se a todos os Escravos em paralelo.
- O Mestre usa uma **linha CS/SS separada** para cada Escravo.
- Ao colocar a ”0” um pino CS, o Mestre "acorda" um Escravo específico para a comunicação, enquanto os outros permanecem a "dormir".

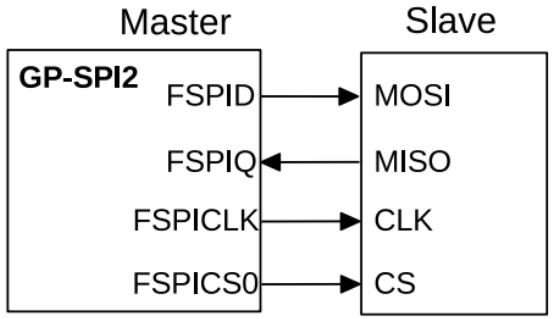


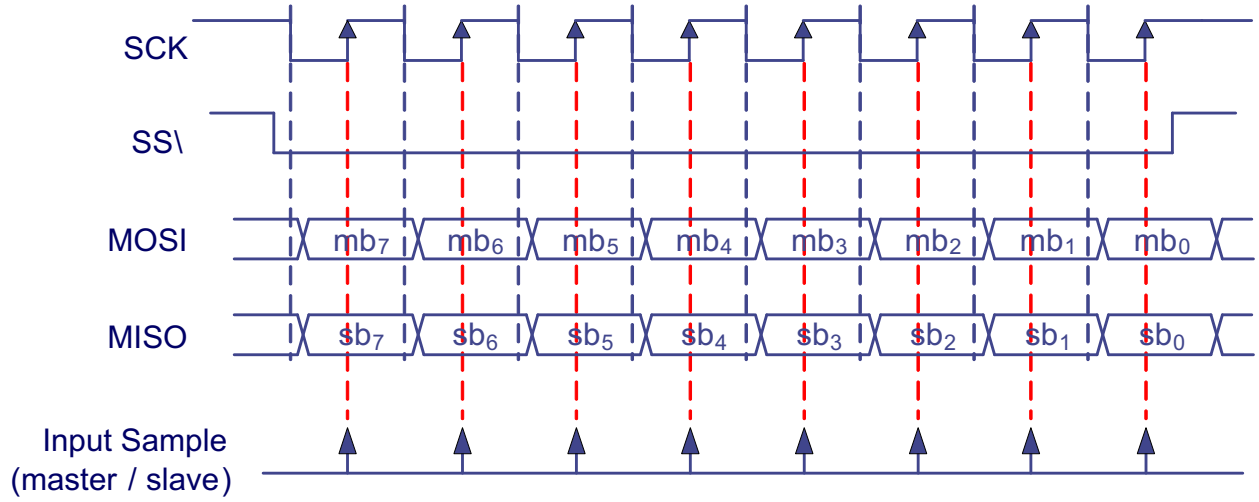
Sinais SPI Full Duplex

Sinal	Nome Completo	Função
SCLK	Serial CLocK (Relógio Serial)	É o "coração" da comunicação. É sempre gerado pelo Mestre (o ESP32-C6) e usado para sincronizar a transferência de dados. Um bit de dados é transferido a cada ciclo do relógio.
MOSI	Master Output, Slave Input	O Mestre envia dados para o Escravo. É o pino de saída de dados do ESP32-C6 e entrada de dados do periférico.
MISO	Master Input, Slave Output	O Mestre recebe dados do Escravo. É o pino de entrada de dados do ESP32-C6 e saída de dados do periférico.
CS/SS	Chip Select / Slave Select (Seleção de Chip)	É gerado pelo Mestre (o ESP32-C6) e específico para cada Escravo. É usado para selecionar qual periférico na linha deve estar ativo.

Operação SPI Full Duplex

- Neste modo, o **mestre SPI fornece sinais CLK (SCK) e CS (SS)**, trocando dados com o escravo SPI no modo de 1 bit através de MOSI (FSPID, envio) e MISO (FSPIQ, receção) ao mesmo tempo.
- O comportamento dos estados CMD, ADDR, DUMMY, **DOUT** e **DIN** é configurável.
- Normalmente, os estados CMD, ADDR e DUMMY não são utilizados nesta comunicação.





universidade de aveiro

SPI Half-Duplex

- O controlador GP-SPI2 no ESP32-C6 (como em muitos dispositivos modernos) suporta modos de comunicação mais flexíveis, incluindo o Half-Duplex
- A inclusão do suporte a **Half-Duplex oferece flexibilidade** para comunicar com diferentes tipos de periféricos e protocolos que não exigem a comunicação bidirecional simultânea.
- Para operar em **Half-Duplex**, o GP-SPI2 é frequentemente configurado para o que é conhecido como **Modo 3-Fios ou modo bidirecional de pino único** (single-wire bidirectional):
- O pino que seria **o MOSI (ou MISO) é configurado como um único pino de I/O,** alternando a sua direção (entrada ou saída) conforme o Mestre (o ESP32-C6) necessite enviar ou receber dados.
- Uma transação Half-Duplex ocorre em duas fases sequenciais sob o controlo do Mestre:
  - **Fase de Transmissão**: O Mestre envia dados (Comando ou Endereço) para o Escravo.
  - **Fase de Receção**: O Mestre reconfigura o pino de dados para entrada e o Escravo começa a enviar a resposta.

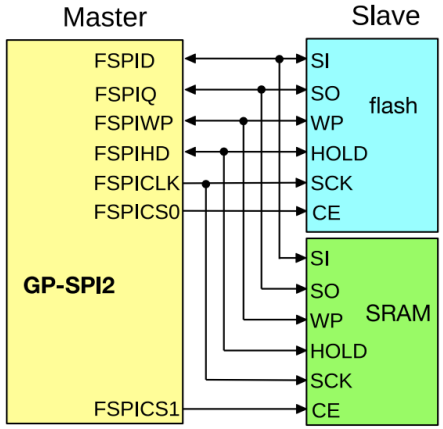
- Este modo é **útil para economizar pinos GPIO**, especialmente em chips menores como o ESP32-C6. É também o modo exigido por certos periféricos que foram projetados para essa configuração mais simples de 3-fios (SCLK, CS e Dados Únicos).

SPI Half-Duplex

Sinal	Nome Comum no ESP32-C6	Função no Modo Half-Duplex
SCLK	Serial CLocK (Relógio Serial)	Gerado pelo Mestre (ESP32-C6) para sincronizar todos os bits de dados. O Escravo não envia dados sem um ciclo de relógio.
DATA/IO	I/O (Input/Output) ou MOSI Reconfigurado	Linha de dados única. Este pino é configurado para ser alternadamente: uma Saída (Output) para o Mestre (envio de comandos/endereços) e uma Entrada (Input) para o Mestre (receção de dados/resposta).
CS/SS	Chip Select / Slave Select (Seleção de Chip)	Gerado pelo Mestre para selecionar o dispositivo Escravo que está ativo e pronto a comunicar. Deve ser mantido ativo (baixo) durante toda a transação de envio e receção.

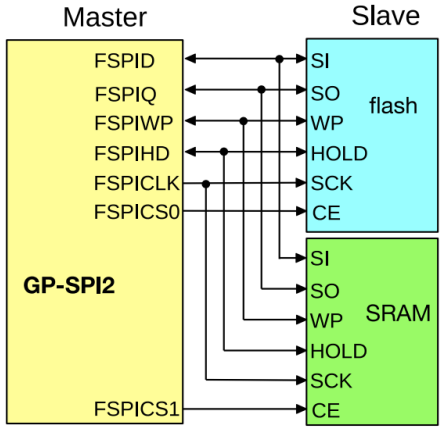
Operação SPI Half Duplex

- **Apenas um lado (SPI mestre ou escravo) pode enviar dados de cada vez,** enquanto o outro lado recebe os dados.
- O formato padrão da comunicação half-duplex SPI é CMD + [ADDR +] [DUMMY +] [DOUT ou DIN]. **Os estados ADDR, DUMMY, DOUT e DIN são opcionais** e podem ser desativados ou ativados independentemente.
- As **propriedades** dos estados GP-SPI2 tais como comprimento do ciclo, valor e modo de bits do barramento paralelo, podem ser **definidas independentemente**.



Operação SPI Half Duplex

- **CMD**: 0 ~ 16 bits, master output, slave input.
- **ADDR**: 0 ~ 32 bits, master output, slave input.
- **DUMMY**: 0 ~ 256 FSPICLK cycles, master output, slave input.
- **DOUT**: 0 ~ 512 bits (64 B) in CPU-controlled mode and 0 ~ 256 Kbits (32 KB) in DMA-controlled mode, master output, slave input.
- **DIN**: 0 ~ 512 bits (64 B) in CPU-controlled mode and 0 ~ 256 Kbits (32 KB) in DMA-controlled mode, master input, slave output.



universidade de aveiro

Sinais usados nos vários modos SPI

FSPI Signal	Master						Slave					
	1-bit SPI			2-bit Dual SPI	4-bit Quad SPI	QPI	1-bit SPI			2-bit Dual SPI	4-bit Quad SPI	QPI
	FD[1]	3-line HD[2]	4-line HD				FD	3-line HD	4-line HD			
FSPICLK	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
FSPICS0	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
FSPICS1	Y	Y	Y	Y	Y	Y						
FSPICS2	Y	Y	Y	Y	Y	Y						
FSPICS3	Y	Y	Y	Y	Y	Y						
FSPICS4	Y	Y	Y	Y	Y	Y						
FSPICS5	Y	Y	Y	Y	Y	Y						
FSPID	Y	Y	(Y)[3]	Y[4]	Y[5]	Y	Y	Y	(Y)[6]	Y[7]	Y[8]	Y
FSPIQ	Y		(Y)[3]	Y[4]	Y[5]	Y	Y		(Y)[6]	Y[7]	Y[8]	Y
FSPIWP					Y[5]	Y					Y[8]	Y
FSPIHD					Y[5]	Y					Y[8]	Y

[1] FD: full-duplex
[2] HD: half-duplex
[3] Only one of the two signals is used at a time.
[4] The two signals are used in parallel.
[5] The four signals are used in parallel.
[6] Only one of the two signals is used at a time.
[7] The two signals are used in parallel.
[8] The four signals are used in parallel.

Kahoot! Time

Inter-Integrated Circuit (I2C)

- O ESP32-C6 possui um controlador I²C principal que pode ser configurado para operar como **Mestre ou Escravo**.
- O **I²C usa apenas duas linhas**: **SDA** (Serial Data) - Linha de dados bidirecional e **SCL** (Serial Clock) - Linha de clock (relógio) gerada pelo Mestre.
- Ambas as linhas (SDA e SCL) **requerem resistências de pull-up externas**. Isso garante que as linhas alcancem um nível lógico *High* quando os dispositivos não estão a transmitir.
  - Apesar do ESP32-C6 possuir *pull-ups* internos, estes são frequentemente insuficientes para uma comunicação I²C fiável.
- Suporta os modos ***Standard*** (até 100 kHz) e ***Fast*** (até 400 kHz), o que é ideal para a maioria dos sensores e *displays*.

Operação do I2C

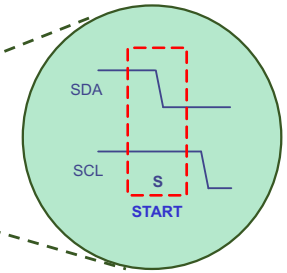
O I²C é um protocolo orientado a endereço e segue um processo bem definido para cada transação

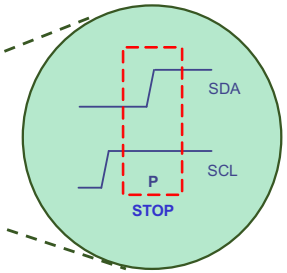
Passo	Ação	Descrição
1	START Condition	O Mestre inicia a comunicação ao forçar a linha SDA para LOW enquanto SCL permanece HIGH.
2	Endereçamento	O Mestre envia um endereço de 7 bits (ou 10 bits) do Escravo que pretende contactar, seguido por um bit R/W (Read/Write - Leitura/Escrita) para indicar a direção da transação.
3	ACK/NACK	O Escravo com o endereço correspondente responde forçando a linha SDA para LOW (chamado ACK - Acknowledge). Se nenhum Escravo responder, o Mestre recebe um NACK e cancela a transação.
4	Transferência de Dados	Os dados são transferidos byte a byte. Após cada byte, o recetor (Mestre ou Escravo) envia um bit ACK para confirmar a receção bem-sucedida.
5	STOP Condition	O Mestre termina a transação forçando a linha SDA para HIGH enquanto SCL permanece HIGH.

Operação do I2C

O I²C é um protocolo orientado a endereço e segue um processo bem definido para cada transação

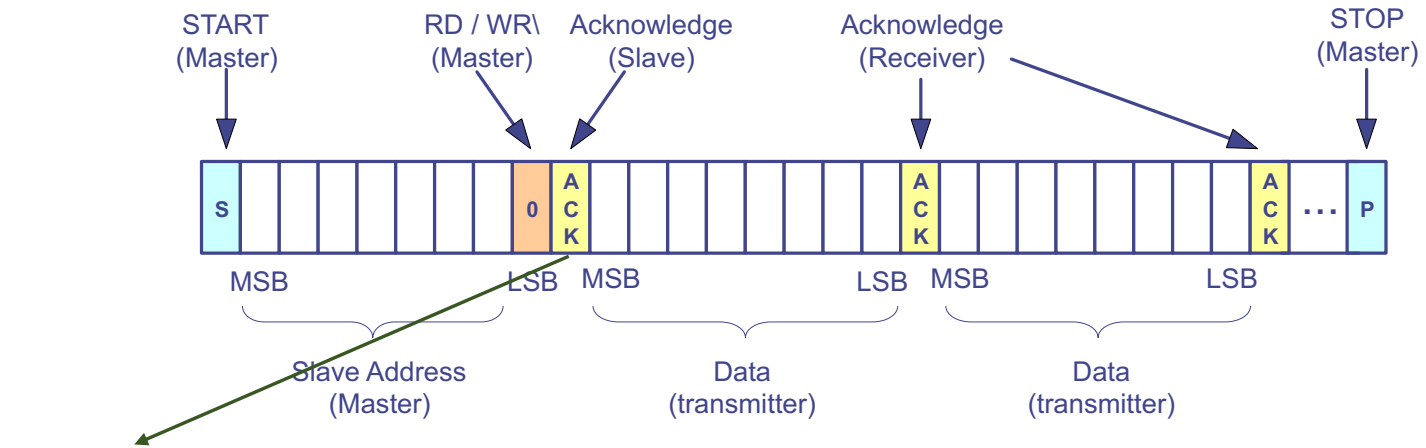
Passo	Ação	Descrição
1	**START** Condition	O Mestre inicia a comunicação ao forçar a linha SDA para LOW enquanto SCL permanece HIGH.
2	Endereçamento	O Mestre envia um endereço de 7 bits (ou 10 bits) do Escravo que pretende contactar, seguido por um bit R/W (Read/Write - Leitura/Escrita) para indicar a direção da transação.
3	ACK/NACK	O Escravo com o endereço correspondente responde forçando a linha SDA para LOW (chamado ACK - Acknowledge). Se nenhum Escravo responder, o Mestre recebe um NACK e cancela a transação.
4	Transferência de Dados	Os dados são transferidos byte a byte. Após cada byte, o recetor (Mestre ou Escravo) envia um bit ACK para confirmar a receção bem-sucedida.
5	**STOP** Condition	O Mestre termina a transação forçando a linha SDA para HIGH enquanto SCL permanece HIGH.





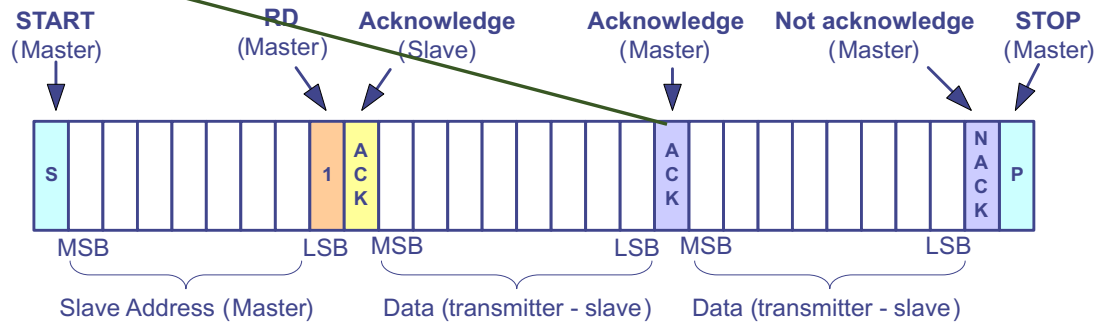
Operação do I2C

Transferência de dados – escrita



Bit dominante!

Transferência de dados – leitura



universidade de aveiro

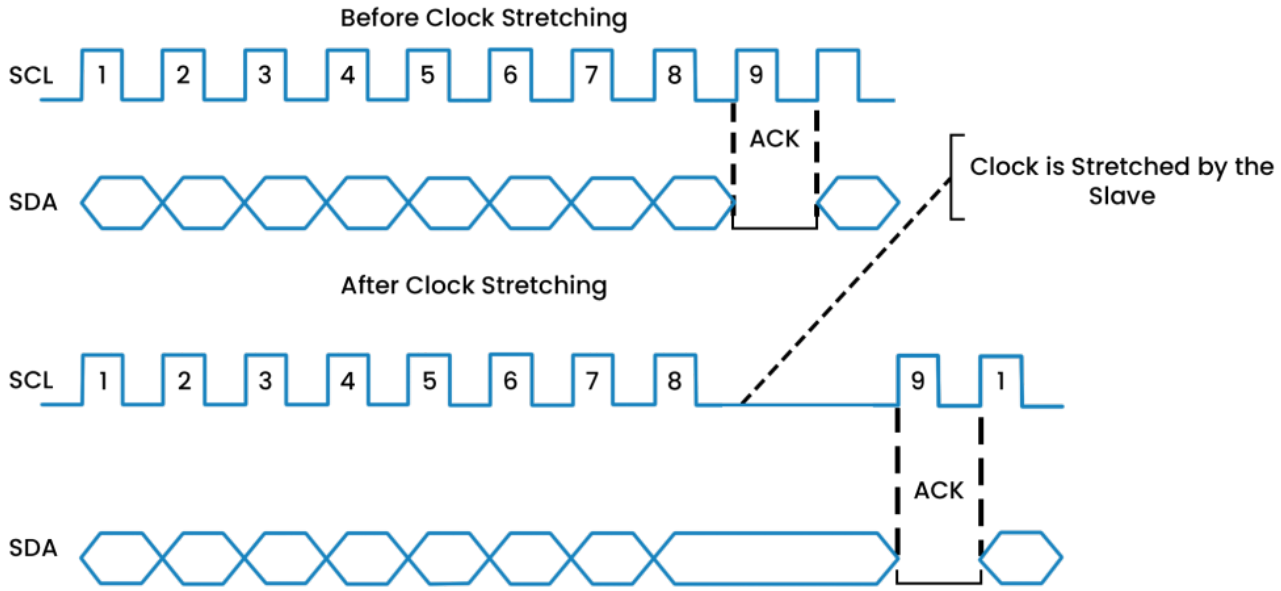
I2C Clock Stretching

- O **Clock Stretching** (ou "Alongamento do Clock") é um mecanismo do protocolo I²C que permite a um dispositivo Escravo controlar temporariamente a velocidade da comunicação.
- Numa comunicação I²C **o Mestre determina a velocidade do clock (SCL).**
  - O Escravo pode ter de executar tarefas internas complexas, como calcular um valor, ler uma memória Flash ou inicializar um periférico, que demoram mais tempo do que o intervalo entre dois ciclos de relógio do Mestre.
  - Se o Mestre continuar a enviar ou a pedir dados antes de o Escravo estar pronto, **a comunicação falhará** ou resultará em dados incorretos.
- O **Clock Stretching ocorre mais frequentemente durante o bit ACK** (Acknowledge) de um byte, quando o Escravo precisa de tempo para processar o byte anterior (ou o endereço) e preparar-se para enviar ou receber o próximo byte.
- Quando o ESP32-C6 está configurado **como Escravo I²C** tem a capacidade de usar o Clock Stretching para garantir que o seu software **tenha tempo suficiente para processar interrupções** e mover os dados para o buffer antes que o Mestre continue.
- Se o ESP32-C6 for o Mestre, é **responsável por suportar e respeitar o Clock Stretching** de qualquer Escravo que o utilize.

I2C Clock Stretching

- O Clock Stretching usa o facto de as **linhas I²C serem Open-Drain** (Dreno Aberto) com resistências de *pull-up*, o que significa que qualquer dispositivo no barramento pode puxar a linha para o nível lógico LOW (Baixo).
- O Mestre **completa um ciclo de relógio** (ou byte de dados) e está prestes a iniciar o próximo ciclo, **forçando a linha SCL para HIGH**.
- O **Escravo que precisa de tempo extra simplesmente mantém a linha SCL em LOW** (Baixo) ativamente.
- Como o Mestre também tenta elevar a linha SCL para HIGH, a lógica Wired-AND garante que, se o Escravo a mantiver em LOW, a linha SCL permanece em LOW.
- **O Mestre monitoriza a linha SCL**. Vendo que a linha está em LOW (apesar de ter tentado subi-la), o **Mestre pausa toda a comunicação e espera**.
- Assim que o Escravo termina o seu processamento interno, liberta a linha SCL.
- As resistências de pull-up puxam SCL de volta para HIGH, o Mestre deteta isso e **retoma a comunicação** (iniciando o próximo ciclo de clock).
- O período em que o Escravo mantém o SCL em LOW é o tempo de clock stretching.

I2C Clock Stretching



universidade de aveiro

Kahoot! Time

universidade de aveiro  theoria poiesis praxis